

pg_stat_kcache

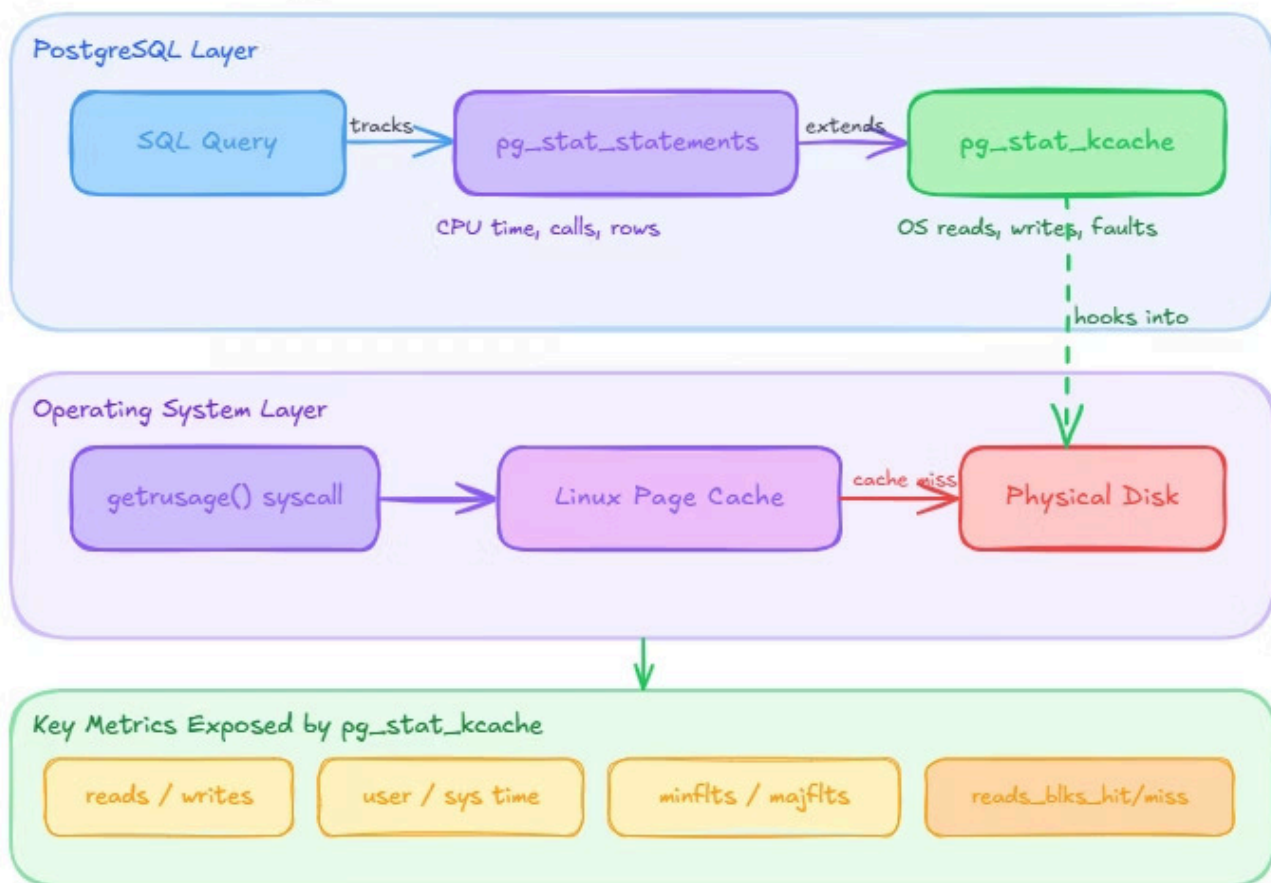
A comprehensive reference for PostgreSQL DBAs and systems engineers on leveraging kernel-level I/O and CPU visibility. This guide covers installation, configuration, core views, and real-world diagnostic scenarios using `pg_stat_kcache` to bridge the gap between database statistics and OS-level resource consumption.

POSTGRESQL EXTENSION

LINUX / UNIX

PERFORMANCE ENGINEERING

pg_stat_kcache - How It Works



What is pg_stat_kcache?

`pg_stat_kcache` is a PostgreSQL extension that surfaces kernel-level resource statistics — CPU usage, physical disk I/O, and block-level access — directly within the PostgreSQL statistics framework. Without it, a DBA can see logical I/O through `pg_stat_statements` or `pg_stat_database`, but has no visibility into how many of those reads actually hit physical storage versus the OS page cache. That blind spot can make query tuning misleading and capacity planning inaccurate.

The extension hooks into POSIX `getrusage()` system calls, capturing resource consumption at statement boundaries. This means every time a query completes, `pg_stat_kcache` records not just wall-clock time but real user-space CPU cycles, kernel-space CPU cycles, physical bytes read and written, soft page faults, and hard page faults. This data is then aggregated at both the database level and the individual query level, giving DBAs a full-stack picture of resource usage.

CPU Time Tracking

User time vs. system/kernel time per query and per database, enabling precise CPU cost attribution.

Physical Disk I/O

Real reads and writes at the filesystem layer — not just `shared_buffers` activity but true storage throughput.

Real Hit Ratio

Calculates the true cache hit ratio including OS page cache, revealing how much I/O bypasses disk entirely.

Query Granularity

Detailed per-statement statistics via `pg_stat_kcache_detail`, linked to `pg_stat_statements` by `queryid`.

Installation Prerequisites

Before installing `pg_stat_kcache`, your environment must satisfy several hard requirements. The extension supports PostgreSQL 9.4 through 17, but it is strictly limited to Linux and other POSIX-compliant Unix systems — Windows is explicitly not supported because it relies on `getrusage()`, which is unavailable on that platform. Most production PostgreSQL deployments on Linux or BSD-based systems will work without issue.

The most critical prerequisite is `pg_stat_statements`. This extension is a hard dependency: `pg_stat_kcache` links kernel statistics to individual queries using the `queryid` generated by `pg_stat_statements`. Without it, query-level granularity is impossible. Both extensions must be declared in `shared_preload_libraries` in `postgresql.conf` before the server starts — they cannot be loaded dynamically at runtime.

PostgreSQL Version

Requires PostgreSQL 9.4 or later. Full planning statistics support requires PG 13+. Latest stable (2.3.0) targets PG 13–17.

Operating System

Linux or Unix (POSIX-compliant) only. Windows is not supported. Relies on `getrusage()` POSIX system call.

`pg_stat_statements`

Must be installed first. It is a mandatory dependency for `queryid`-level tracking and query-level statistics correlation.

Installation: Step-by-Step

Installation requires both a server-side configuration change and a SQL-level extension creation. The order of operations matters: `shared_preload_libraries` must be set and the server restarted before the `CREATE EXTENSION` command will succeed. Skipping the restart and attempting the SQL command directly will result in an error indicating the module is not loaded.

1 Configure `shared_preload_libraries`

Add both extensions to `postgresql.conf` before starting the server. Both entries are required on the same line:

```
shared_preload_libraries
=
'pg_stat_statements,pg_stat_kcache'
```

Restart PostgreSQL after this change. Without a full server restart, the modules will not be loaded into shared memory.

2 Install the Extension via SQL

Two installation methods are available. The `CASCADE` method is preferred as it auto-installs the dependency:

```
-- Method 1: CASCADE (recommended)
CREATE EXTENSION
pg_stat_kcache CASCADE;

-- Method 2: Manual dependency order
CREATE EXTENSION
pg_stat_statements;
CREATE EXTENSION
pg_stat_kcache;
```

3 Verify Installation

Confirm both extensions appear in the extension catalog:

```
\dx

Name | Version |
Schema | Description
-----+-----+-----
--
pg_stat_kcache | 2.3.0 |
public | Kernel statistics
gathering
pg_stat_statements | 1.11
| public | track planning
and execution statistics
```

Configuration Parameters

Once installed, three GUC (Grand Unified Configuration) parameters control the behavior of `pg_stat_kcache`. These can be set in `postgresql.conf`, via `ALTER SYSTEM`, or at the session level using `SET`. Understanding them is critical for accurate data collection, particularly in high-frequency sampling environments or when analyzing stored procedure performance.

The `linux_hz` parameter deserves special attention. PostgreSQL's kernel statistics depend on the Linux kernel's `CONFIG_HZ` tick rate, which controls how frequently the scheduler updates CPU time accounting. If this value is misconfigured or left at `-1`, the extension will attempt to auto-detect it — which works correctly on most modern kernels but may produce slight inaccuracies on custom or embedded kernel builds. For production environments where sub-millisecond CPU accounting precision matters, explicitly setting this to your kernel's actual tick rate (typically 100, 250, or 1000 Hz) is recommended.

Parameter	Type	Default	Description
<code>pg_stat_kcache.linear_hz</code>	integer	-1	Linux <code>CONFIG_HZ</code> value for sampling error compensation. -1 enables auto-detection.
<code>pg_stat_kcache.track</code>	enum	top	Track <code>top</code> (client statements only), <code>all</code> (includes nested/procedural), or <code>none</code> (disabled).
<code>pg_stat_kcache.track_planning</code>	boolean	off	Track planning phase CPU and I/O separately. Requires PostgreSQL 13 or later.

- Setting `track = 'all'` will capture statistics for nested statements inside PL/pgSQL functions and procedures. This increases overhead slightly but is essential when diagnosing CPU usage inside stored routines.

Core View: pg_stat_kcache

The `pg_stat_kcache` view provides database-level aggregations of all kernel statistics collected since the last reset. It is the primary entry point for fleet-level analysis — ideal for capacity planning, identifying resource-heavy databases in a multi-tenant cluster, or establishing a baseline before and after a major schema migration.

Each row in this view corresponds to one database. The columns are organized into planning-phase metrics (prefixed `plan_`) and execution-phase metrics (prefixed `exec_`). Note that planning metrics are only populated when `pg_stat_kcache.track_planning = on` and PostgreSQL 13+ is in use. The `stats_since` column records when statistics collection began for each database, which is critical for interpreting cumulative values correctly.

Column	Type	Description
<code>datname</code>	<code>name</code>	Database name
<code>plan_user_time</code>	double precision	User CPU time consumed during planning (seconds)
<code>plan_system_time</code>	double precision	System/kernel CPU time consumed during planning
<code>plan_reads_blks</code>	bigint	8K blocks read from disk during planning phase
<code>plan_writes_blks</code>	bigint	8K blocks written during planning phase
<code>exec_user_time</code>	double precision	User CPU time consumed during execution (seconds)
<code>exec_system_time</code>	double precision	System CPU time consumed during execution
<code>exec_reads</code>	bigint	Bytes read by filesystem during execution
<code>exec_reads_blks</code>	bigint	8K blocks read (physical I/O during execution)
<code>exec_writes</code>	bigint	Bytes written during execution
<code>exec_writes_blks</code>	bigint	8K blocks written during execution
<code>exec_minflts</code>	bigint	Soft (minor) page faults — pages satisfied from memory
<code>exec_majflts</code>	bigint	Hard (major) page faults — pages requiring disk reads
<code>stats_since</code>	timestamptz	Timestamp when statistics collection began for this database

Core View: pg_stat_kcache_detail

While `pg_stat_kcache` aggregates by database, `pg_stat_kcache_detail` provides per-query granularity. Each row corresponds to a unique combination of `queryid`, `userid`, and `dbid` — the same triple key used by `pg_stat_statements`. This alignment is by design: joining these two views on `queryid` gives you a complete picture of a query's resource footprint, combining logical buffer statistics from `pg_stat_statements` with physical I/O and CPU data from `pg_stat_kcache_detail`.

The `top` boolean column distinguishes between top-level client statements and nested calls. When `pg_stat_kcache.track = 'all'`, nested statements (such as those called from within PL/pgSQL functions) are tracked separately with `top = false`. For most OLTP analysis, filtering on `top = true` avoids double-counting.

Key Identity Columns

- `queryid` — bigint, links to `pg_stat_statements`
- `userid` — oid, the executing user
- `dbid` — oid, the target database
- `top` — boolean, true for client-facing statements

Metric Columns

All `plan_*` and `exec_*` columns from `pg_stat_kcache` are present here at per-query resolution. This includes user/system CPU seconds, physical block reads/writes, byte-level I/O, and page fault counters — identical schema, per-statement scope.

Use `pg_stat_kcache_reset()` to clear all accumulated data and establish a fresh baseline.

Scenario 1: Identify Top CPU-Consuming Databases

The most immediate use case for `pg_stat_kcache` is identifying which databases in a cluster are generating the most CPU load. Unlike `pg_stat_activity`, which shows instantaneous active queries, `pg_stat_kcache` provides cumulative CPU seconds — user-space time (application logic, sort operations, aggregations) and system-space time (kernel calls, I/O scheduling, memory management) — giving a historical picture of where CPU resources are being spent.

The following query ranks databases by total CPU consumption. The separation of user and system time is analytically valuable: a database with disproportionately high system time relative to user time often indicates heavy I/O activity, frequent context switches, or memory pressure — all of which are OS-level symptoms rather than query-logic problems.

```
SELECT
  datname,
  ROUND(exec_user_time::numeric, 3) AS user_cpu_sec,
  ROUND(exec_system_time::numeric, 3) AS sys_cpu_sec,
  ROUND((exec_user_time + exec_system_time)::numeric, 3) AS total_cpu_sec,
  exec_reads_blks,
  exec_writes_blks
FROM pg_stat_kcache
ORDER BY (exec_user_time + exec_system_time) DESC;
```

- ❏ A high **sys_cpu_sec / user_cpu_sec ratio** (greater than ~0.3) is a strong signal of I/O bottlenecks or kernel-level contention. Investigate with `iostat` and `vmstat` alongside this query output.

Scenario 2: Find I/O-Intensive Queries

Physical I/O is among the most expensive operations in any database workload. While `pg_stat_statements` tracks `shared_blks_read` (logical buffer misses that were served from `shared_buffers` or the OS page cache), only `pg_stat_kcache` reveals how many of those accesses actually hit the physical disk — bypassing both PostgreSQL's `shared_buffers` and the OS page cache entirely. This distinction is critical for index analysis, as sequential scans on large tables generate far more physical I/O than index range scans on well-cached data.

The query below joins `pg_stat_kcache_detail` with `pg_stat_statements` to correlate physical read volume with the actual SQL text. The filter on `exec_reads_blks > 1000` targets queries that have caused more than 8 MB of physical disk reads, and the result is sorted by read intensity to surface the worst offenders first.

```
SELECT
  d.queryid,
  s.query,
  d.exec_reads_blks,
  d.exec_writes_blks,
  ROUND(d.exec_user_time::numeric, 3) AS cpu_sec,
  ROUND((d.exec_reads_blks * 8.0 / 1024)::numeric, 2) AS read_mb
FROM pg_stat_kcache_detail d
JOIN pg_stat_statements s ON d.queryid = s.queryid
WHERE d.exec_reads_blks > 1000
ORDER BY d.exec_reads_blks DESC
LIMIT 10;
```

→ Identify the Query

Surface queries with `exec_reads_blks > 1000` (8+ MB physical reads) using the join above.

→ Analyze the Plan

Run `EXPLAIN (ANALYZE, BUFFERS)` on the query to identify sequential scans and missing index opportunities.

→ Create Index & Measure

After adding an index, reset with `pg_stat_kcache_reset()` and re-run the workload. Compare new `exec_reads_blks` to confirm I/O reduction.

Scenario 3: Calculate the Real Cache Hit Ratio

The standard PostgreSQL cache hit ratio — derived from `blks_hit / (blks_hit + blks_read)` in `pg_stat_database` — only reflects `shared_buffers` hits versus misses. It does not account for the OS page cache, which typically absorbs a large fraction of the "missed" `shared_buffers` reads before they reach physical storage. This means the traditional metric can significantly understate true cache effectiveness, leading DBAs to over-provision `shared_buffers` unnecessarily.

`pg_stat_kcache` solves this by providing `exec_reads_blks`, which represents blocks that actually went all the way to the physical device. Combined with total logical reads from `pg_stat_database`, this enables a three-tier hit ratio calculation: `shared_buffers` hit, OS page cache hit, and true physical disk access.

```
WITH stats AS (
  SELECT
    datname,
    exec_reads_blks AS physical_reads,
    (SELECT sum(blks_read + blks_hit)
     FROM pg_stat_database d2
     WHERE d2.datname = pg_stat_kcache.datname) AS logical_reads
  FROM pg_stat_kcache
)
SELECT
  datname,
  physical_reads,
  logical_reads,
  CASE
    WHEN logical_reads > 0
    THEN ROUND(((logical_reads - physical_reads)::numeric
                / logical_reads * 100), 2)
    ELSE 0
  END AS true_cache_hit_pct,
  ROUND((physical_reads * 8.0 / 1024)::numeric, 2) AS physical_read_mb
FROM stats
ORDER BY physical_reads DESC;
```

- ❏ A **true_cache_hit_pct** above 99% indicates that OS page cache is effectively absorbing most I/O misses. If this drops below 95% under normal load, consider increasing the system's available RAM or tuning `effective_cache_size` in the planner configuration.

Scenario 4: Detect Planning vs. Execution Bottlenecks

In most OLTP workloads, query execution dominates CPU time and planning overhead is negligible. However, in systems with complex schemas, many joins, or heavy use of stored procedures that re-plan on every invocation, planning CPU cost can become a measurable fraction of total query cost. Identifying these cases is only possible when `pg_stat_kcache.track_planning = on` is configured in `postgresql.conf`, which requires PostgreSQL 13 or later.

When planning costs are elevated — typically when planning time exceeds 10% of execution time — the remediation strategies differ entirely from execution-side tuning. Instead of adding indexes or rewriting SQL, the focus shifts to increasing `plan_cache_mode`, reviewing `geqo_threshold`, reducing plan invalidation frequency, or using prepared statements to amortize planning cost across multiple executions.

```
SELECT
  datname,
  ROUND(plan_user_time::numeric, 4) AS plan_user_sec,
  ROUND(plan_system_time::numeric, 4) AS plan_sys_sec,
  ROUND(exec_user_time::numeric, 4) AS exec_user_sec,
  ROUND(exec_system_time::numeric, 4) AS exec_sys_sec,
  CASE
    WHEN (plan_user_time + plan_system_time)
      > (exec_user_time + exec_system_time) * 0.1
    THEN 'HIGH_PLANNING_COST'
    ELSE 'NORMAL'
  END AS plan_cost_flag
FROM pg_stat_kcache
WHERE pg_stat_kcache.track_planning = true
ORDER BY (plan_user_time + plan_system_time) DESC;
```

HIGH_PLANNING_COST Remediation

- Use prepared statements to cache plans
- Set `plan_cache_mode = force_generic_plan`
- Reduce join complexity in problematic queries
- Review `geqo_threshold` for large join graphs

When to Enable `track_planning`

- Workloads with heavy ad-hoc query patterns
- Systems using ORM frameworks that re-prepare queries
- Stored procedure-heavy applications
- After upgrading to PostgreSQL 13+ for baseline data

Index Performance Analysis

`pg_stat_kcache` is especially powerful for index optimization workflows because it measures the actual physical cost of missing or inefficient indexes — not just logical buffer activity. When a sequential scan occurs on a large table, it generates a burst of physical reads that are clearly visible in `exec_reads_blks`. After an appropriate index is created and the planner switches to an index scan, those physical reads drop dramatically, and the improvement is immediately quantifiable.

Write amplification is another critical index-related metric. Every INSERT or UPDATE on an indexed table must maintain each index's B-tree structure, generating additional write I/O beyond the heap tuple itself. High `exec_writes_blks` on tables with many indexes — especially partial or expression indexes — can indicate that write amplification is a meaningful contributor to I/O load, which may warrant consolidating or dropping underused indexes.

-- Correlate physical I/O with buffer hit ratio to find indexing candidates

```
SELECT
  k.queryid,
  LEFT(s.query, 100)    AS query_preview,
  k.exec_reads_blks,
  k.exec_user_time,
  k.exec_system_time,
  s.shared_blks_hit,
  s.shared_blks_read
FROM pg_stat_kcache_detail k
JOIN pg_stat_statements s ON k.queryid = s.queryid
WHERE k.exec_reads_blks > 100
      AND s.shared_blks_read > s.shared_blks_hit * 0.5
ORDER BY k.exec_reads_blks DESC
LIMIT 20;
```



Sequential Scan Detection

High `exec_reads_blks` combined with low `shared_buffers` hit ratio indicates missing indexes driving full table scans.



Index Efficiency Measurement

Compare `exec_reads_blks` before and after index creation. Use `pg_stat_kcache_reset()` to establish a clean baseline between tests.



Write Amplification Detection

Elevated `exec_writes_blks` relative to row modification volume signals heavy index maintenance overhead during writes.

Integration with PoWA

PoWA (PostgreSQL Workload Analyzer) is a companion monitoring framework that collects periodic snapshots of `pg_stat_statements`, `pg_stat_kcache`, and other extensions, storing historical time-series data in a dedicated database. When `pg_stat_kcache` is registered with PoWA, its metrics become available as time-series graphs in the PoWA web UI, enabling trending and anomaly detection that a point-in-time query against the live views cannot provide.

Registration is a one-time SQL call executed against the PoWA repository database. After registration, PoWA's background worker will automatically include `pg_stat_kcache` snapshots in its collection cycle. This integration unlocks three enhanced graph types in the PoWA query analysis page: physical block I/O over time, CPU time split between user and system percentages, and an enhanced hit ratio graph that separately tracks `shared_buffers` hits, OS page cache hits, and true disk reads — providing the three-tier visibility described in Scenario 3.

```
-- Register pg_stat_kcache with PoWA
SELECT powa_kcache_register();
```



Physical Block Graphs

Time-series visualization of physical read and write block counts per query, enabling I/O trend analysis across workload windows.



CPU Time Metrics

Historical user vs. system CPU time percentage breakdown, useful for identifying when CPU profile shifts correlate with schema or workload changes.

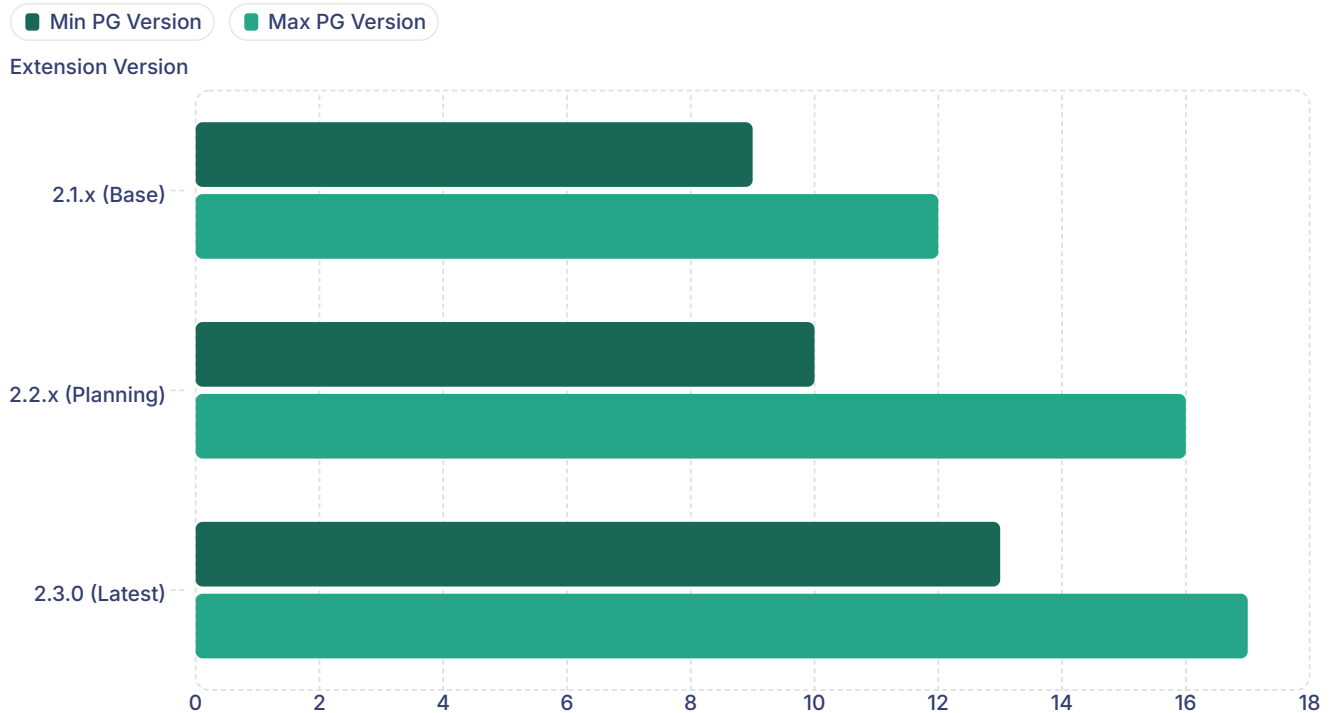


Enhanced Hit Ratio

Three-tier hit ratio graph showing `shared_buffers`, OS page cache, and disk read proportions over time — far more precise than standard buffer hit ratio metrics.

Version Compatibility

The `pg_stat_kcache` project follows a regular release cadence aligned with PostgreSQL major version support. Each version series introduces capability improvements and drops support for older PostgreSQL releases as they reach end-of-life. Understanding which version you need is important: deploying an older extension version on a newer PostgreSQL major version may result in missing columns, reduced metric accuracy, or compilation failures.



Version	PostgreSQL Support	Key Features
2.3.0	PostgreSQL 13–17	Latest stable release. Full planning stats, all exec metrics, stats_since column, PG 17 compatibility.
2.2.x	PostgreSQL 10–16	Introduced planning phase statistics (plan_* columns). Requires PG 13+ for track_planning.
2.1.x	PostgreSQL 9.4–12	Base functionality: execution CPU, physical I/O, page faults. No planning metrics.

For the latest source code, changelogs, and issue tracking, refer to the official repository at github.com/powa-team/pg_stat_kcache.

Best Practices for DBAs

Effective use of `pg_stat_kcache` requires more than just running the queries shown above — it demands disciplined operational habits around when to reset statistics, how to combine data sources, and how to interpret cumulative metrics in the context of your workload's age. The following practices are distilled from production deployments and represent the operational baseline recommended for any team using this extension for performance engineering.

1

Always Deploy with `pg_stat_statements`

The dependency is mandatory, but beyond installation, the real value is in joining on `queryid`. Never analyze `pg_stat_kcache_detail` in isolation — always enrich it with `pg_stat_statements` data for full query text, execution count, and total time context.

2

Monitor Overhead Proactively

The overhead of `pg_stat_kcache` is minimal, but test in a staging environment before enabling on high-frequency OLTP clusters exceeding tens of thousands of TPS. The `getrusage()` call adds a small per-statement cost that compounds at extreme query rates.

3

Reset Strategically

Use `SELECT pg_stat_kcache_reset()` after major schema changes, index additions, or bulk load jobs to establish clean baselines. Cumulative statistics spanning architectural changes make before/after comparison impossible. Document every reset with a timestamp and context.

4

Use for Capacity Planning

Track `exec_reads_blks` and `exec_writes_blks` trends over weekly and monthly windows to predict I/O throughput requirements. Combine with storage device throughput specs to project when current hardware will become saturated under projected growth.

5

Enable `track_planning` for Complex Workloads

On PostgreSQL 13+ with ORM-heavy or ad-hoc query workloads, set `pg_stat_kcache.track_planning = on` to surface hidden planning overhead. This is especially important on systems where the same logical query is re-planned frequently due to bind parameter sniffing or plan invalidation.

Learn More & Stay Connected

www.thedbadmin.com

admin@thedbadmin.com

+91 6232090055

Join our WhatsApp & Telegram community for DBA tips, jobs & learning 🚀

WhatsApp Community

Telegram Community